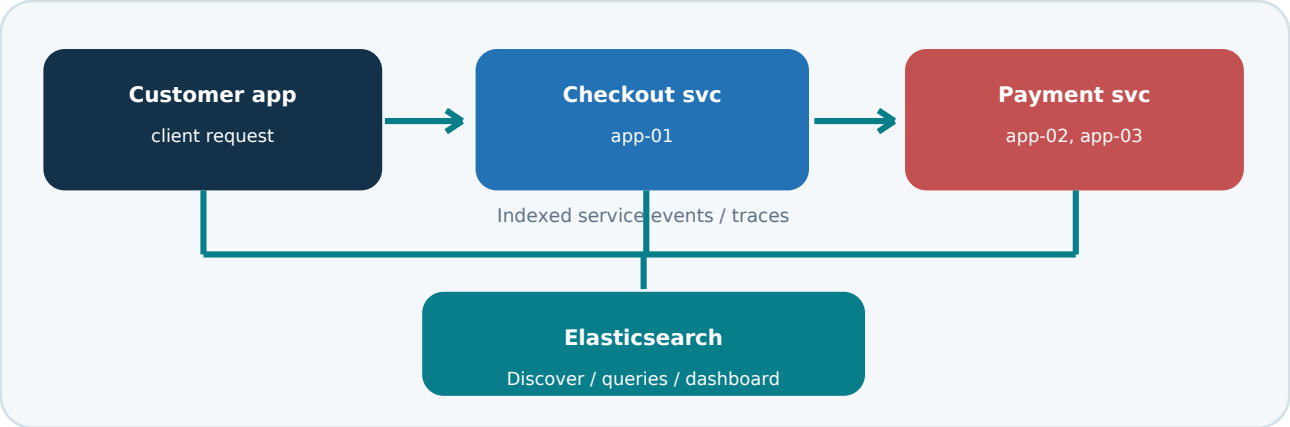
PRACTICAL ELK OBSERVABILITY LAB

Service Incident Query Workshop & Dashboard Validation

A step-by-step investigation, query, visualization, and evidence-based validation manual for Kibana and Elasticsearch



LAB ENVIRONMENT	SCENARIO	DURATION
Ubuntu | 3 Elasticsearch nodes + Kibana	Payment and checkout errors	120 minutes

Workshop result

Participants will create a saved incident dashboard, reproduce its metrics with queries, and submit an evidence-based incident assessment.

Included: index mapping, 12-record training dataset, KQL exercises, ES|QL aggregation, Lens dashboard procedure, validation Query DSL, expected results, assessment rubric, and troubleshooting notes.

Copy-ready commands: companion file Service_Incident_Dev_Tools_Commands.txt. The same dataset is reproduced in tabular form in this PDF.

1. Workshop overview

This lab simulates an application incident. Investigators must distinguish **what the data proves** (five failed requests) from **what still requires investigation** (the underlying root cause).

Learning outcomes

- Locate incident records in Discover with an absolute time range.
- Write KQL filters for failure events, host impact, latency, and trace IDs.
- Summarize failures and failure rates using ES|QL.
- Create service and host visualizations in Lens.
- Independently validate counts, filters, rates, and aggregation results.

Agenda

LAB	ACTIVITY	TIME
Lab 1	Prepare index, sample records, data view	25 min
Lab 2	KQL, ES|QL, trace correlation	30 min
Lab 3	Create operations dashboard	35 min
Lab 4 / capstone	Validation and incident assessment	30 min

Prerequisites and safe working rules

- Kibana and Elasticsearch are accessible; your account has privileges to create an index, data view, and dashboard.
- Use the dedicated **svc-incidents-lab** training index; do not change existing production indices or cluster settings.
- Open **Kibana > Dev Tools > Console** for REST commands and **Discover** for KQL/ES|QL.
- The incident records use UTC. For Chennai/IST, 02:13 UTC is 07:43 IST.
- Menu labels and visualization choices can vary by Elastic Stack version. Use global search for Data Views, Discover, and Dashboards when the menu differs.

Incident brief

A payment backend reports HTTP 503/504 errors. Checkout also reports an HTTP 502. Determine when the incident began, which service and host are most impacted, and whether dashboard counts are trustworthy.

2. Lab 1 - Create the dataset

Step 1. Create the training index

In Kibana Dev Tools Console, execute the mapping command. The fields are explicitly mapped so KQL, Lens, and ES|QL aggregate correctly.

```
PUT svc-incidents-lab
{
  "mappings": {"properties": {
    "@timestamp": {"type": "date"},
    "service": {"properties": {"name": {"type": "keyword"}}},
    "host": {"properties": {"name": {"type": "keyword"}}},
    "event": {"properties": {"outcome": {"type": "keyword"}}},
    "log": {"properties": {"level": {"type": "keyword"}}},
    "http": {"properties": {"response": {"properties": {
      "status_code": {"type": "integer"}
    }}}},
    "trace": {"properties": {"id": {"type": "keyword"}}},
    "response_time_ms": {"type": "integer"}
  }}
}
```

Expected output: `"acknowledged": true`. If the index already exists, verify its mapping first and do not blindly recreate it.

3. Lab 1 - Load and verify records

Step 2. Insert all 12 simulated service events

Paste the `POST svc-incidents-lab/_bulk?refresh=true` block from the companion Dev Tools text file into Kibana Console. The supplied script uses fixed document IDs (1-12) to keep test results repeatable.

> **Important for reproducibility**
>
> Run the complete seed block. Confirm the Bulk API returns `"errors": false`. If any item reports an error, inspect that item before continuing.

Sample event records

#	Time IST	Service	Host	HTTP	ms	Outcome
1	07:40	auth	app-01	200	85	OK
2	07:41	checkout	app-01	200	145	OK
3	07:42	payment	app-02	200	180	OK
4	07:43	payment	app-02	503	1800	FAIL
5	07:44	payment	app-02	503	2400	FAIL
6	07:45	checkout	app-01	502	1600	FAIL
7	07:46	payment	app-02	504	3200	FAIL
8	07:47	checkout	app-01	200	890	OK
9	07:48	payment	app-02	200	920	OK
10	07:49	auth	app-01	200	95	OK
11	07:50	payment	app-03	503	2100	FAIL
12	07:51	checkout	app-01	200	130	OK

Step 3. Verify the source data

```
GET svc-incidents-lab/_count
GET svc-incidents-lab/_mapping
GET svc-incidents-lab/_search?size=3&sort=@timestamp:asc
```

The count must be **12**. The first record should be the auth-service event at **02:10 UTC / 07:40 IST**. All documents must contain the timestamp and expected service/host fields.

Step 4. Create data view and open Discover

- Open Stack Management > Data Views > Create data view (or find Data Views via Kibana search).
- Name: **Service Incident Lab**; index pattern: **svc-incidents-lab**.
- Timestamp field: **@timestamp**. Save the data view.
- Open Discover; select that data view, and set an absolute time window **6 Nov 2026 07:30-08:30 IST** (02:00-03:00 UTC).
- Verify 12 visible records. Add columns: @timestamp, service.name, host.name, event.outcome, http.response.status_code, response_time_ms.

4. Lab 2 - Service incident queries

Exercise A. Run KQL filters in Discover

QUESTION	KQL FILTER	EXPECTED
All failures	event.outcome: "failure"	5
HTTP server errors	http.response.status_code >= 500	5
Payment errors	service.name: "payment-service" AND event.outcome: "failure"	4
Slow requests	response_time_ms > 1000	5
Same trace	trace.id: "trace-pay002"	2
app-02 failures	host.name: "app-02" AND event.outcome: "failure"	3

How to run: select the training data view in Discover, choose KQL / classic search, paste one filter at a time into the search bar, and click Refresh/Update. Record actual count against expected count.

Exercise B. Summarize by service using ES|QL

Switch Discover to ES|QL mode. In this mode you query the index directly, without requiring a data view.

```
FROM svc-incidents-lab
| EVAL is_failure = CASE(event.outcome == "failure", 1, 0)
| STATS requests = COUNT(*),
        failures = SUM(is_failure),
        avg_latency_ms = AVG(response_time_ms)
  BY service.name
| EVAL failure_rate_pct = ROUND(failures * 100.0 / requests, 1)
| SORT failures DESC
```

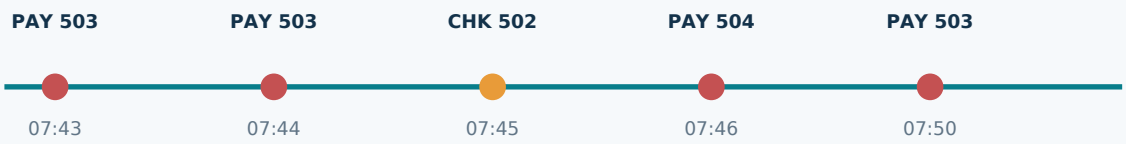
SERVICE	REQUESTS	FAILURES	FAIL RATE
payment-service	6	4	66.7%
checkout-service	4	1	25.0%
auth-service	2	0	0.0%

The percentage is calculated as failures divided by requests. This workshop uses a tiny synthetic sample, so these percentages are examples, not production reliability measurements.

5. Lab 2 - Trace correlation and triage

Exercise C. Reconstruct the incident

Use the absolute time window and sort ascending by @timestamp. Focus on service errors and the linked trace ID.



Time shown in IST. Related trace ID connects the 07:44 and 07:45 events.

Follow the correlated trace

```
trace.id: "trace-pay002"
```

EVENT TIME	SERVICE	HTTP	TRACE ID
07:44 IST	payment-service	503	trace-pay002
07:45 IST	checkout-service	502	trace-pay002

What the data proves

- Payment-service had four failures (three 503s and one 504).
- Checkout-service had one HTTP 502 failure.
- The first failure was 07:43 IST on 6 Nov 2026.
- The same trace ID appears in a payment failure and the subsequent checkout failure.

What remains a hypothesis

A downstream payment dependency issue may have caused the checkout error. **Do not declare root cause from these records alone.** Confirm using APM traces, upstream/downstream span context, deploy changes, resource metrics, and additional application logs.

> **Investigator prompt**
>
> Is the payment issue isolated to app-02? No: one failed payment request also appears on app-03. Does the shared trace establish causal order? Not by itself - it establishes correlation.

Evidence to capture

Screenshot the filtered Discover view; record the absolute time range, query, relevant trace, affected services, and current working hypothesis.

6. Lab 3 - Build the operations dashboard

Step 1. Create a dashboard

- Open **Dashboards > Create dashboard**.
- Select **Create visualization** / **Lens** and the **Service Incident Lab** data view.
- Set global time range to **6 Nov 2026 07:30-08:30 IST**.
- Create and save each panel, then arrange it using the layout shown on the next page.

Step 2. Create required Lens panels

PANEL NAME	VISUAL TYPE	FIELDS / FORMULA
Total requests	Metric	Records count; expected 12
Failed requests	Metric	Records count + KQL filter: event.outcome: "failure"
Failure rate	Metric	Lens formula shown below; Percent formatting
Errors over time	Line	@timestamp on X; count; KQL filter to failures
Failures by service	Horizontal bar	service.name terms; count; failure-only filter
Failures by host	Horizontal bar	host.name terms; count; failure-only filter
HTTP status codes	Donut	http.response.status_code terms, count
Average response ms	Metric	Average of response_time_ms

Step 3. Add the failure-rate formula

```
count(kql='event.outcome: "failure"') / count()
```

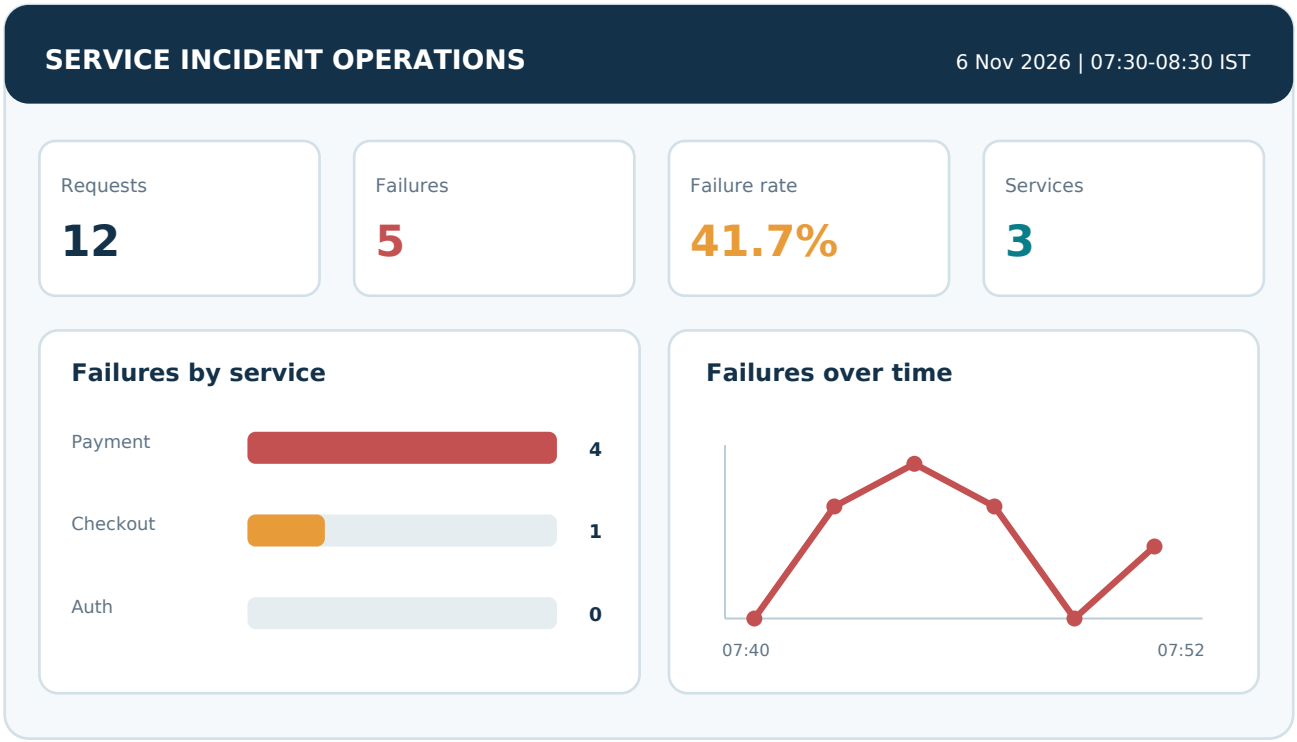
In Lens, select **Formula** as the metric calculation, paste the expression, and choose **Percent** as the value format. Expected result: **41.7%**.

Step 4. Add dashboard controls

Add Options list controls for **service.name** and **host.name**. Save the completed dashboard as **Service Incident Operations Dashboard**. Ensure every panel uses the intended data view and global time range.

7. Dashboard blueprint and expected results

Example layout for the training dataset (illustrative - rendered here as a design guide, not a Kibana screenshot).



Expected dashboard cross-check values

METRIC	ALL SERVICES	PAYMENT-ONLY FILTER
Requests	12	6
Failed requests	5	4
Failure rate	41.7%	66.7%
Mean response time	~1128.8 ms	~1766.7 ms

HTTP code distribution

STATUS	COUNT	INTERPRETATION
200	7	Successful responses, including WARN logs
502	1	Checkout gateway / downstream error
503	3	Payment service unavailable
504	1	Payment gateway timeout

Validation principle: use a consistent time range and filter state. A dashboard number is not considered correct solely because it looks plausible.

8. Lab 4 - Dashboard validation

Step 1. Verify the source-of-truth aggregation

In Dev Tools, run the following Query DSL request. Review the total hit count and each aggregation bucket.

```
GET svc-incidents-lab/_search
{
  "size": 0,
  "query": {
    "range": {
      "@timestamp": {
        "gte": "2026-11-06T02:00:00Z",
        "lt": "2026-11-06T03:00:00Z"
      }
    }
  },
  "aggs": {
    "by_service": {
      "terms": { "field": "service.name", "size": 10 },
      "aggs": {
        "failures": {
          "filter": { "term": { "event.outcome": "failure" } }
        }
      }
    }
  }
}
```

Step 2. Evaluate dashboard test cases

ID	TEST	EXPECTED RESULT	RECORD
V01	Base total	12 requests; 5 failures	Pass / Fail
V02	Payment filter	6 requests; 4 failures	Pass / Fail
V03	Global error rate	5 / 12 = 41.7%	Pass / Fail
V04	Payment error rate	4 / 6 = 66.7%	Pass / Fail
V05	Host breakdown	app-01=1; app-02=3; app-03=1	Pass / Fail
V06	Time consistency	all panels use 07:30-08:30 IST	Pass / Fail
V07	Source comparison	Discover + Dev Tools agree	Pass / Fail

9. Validate behavior, not only values

Test dashboard interactions

- Select **payment-service** using the service control; capture total requests, failures, and calculated failure rate.
- Clear that control; select **app-02**; verify 3 failed requests.
- Set the absolute time range to an unrelated period; incident counts should drop to zero when no records match. Restore the incident window.
- Inspect a panel request (panel menu > Inspect, when available) and compare its filtering and aggregation with the Dev Tools request.
- Check whether a panel uses a pinned filter, a custom panel time range, or a separate data view; each can explain a count mismatch.

Evidence register for participants

ITEM	SCREENSHOT / RECORD	PASS?
E1	Discover view with time range and 12 hits	[ ]
E2	KQL payment-service failure query (4 hits)	[ ]
E3	ES|QL service summary table	[ ]
E4	Saved Lens dashboard (all panels visible)	[ ]
E5	Filtered payment-only dashboard	[ ]
E6	Dev Tools aggregation and matching values	[ ]
E7	Incident assessment / root-cause hypothesis	[ ]

Common validation defects

SYMPTOM	LIKELY CAUSE	FIX
Zero results	Time picker excludes 6 Nov 2026	Choose exact absolute UTC or IST window
Wrong totals	Seed request partially failed or other data present	Check _count, bulk errors, target index
Metric differs from Discover	Different filters or panel-level time range	Inspect all filter chips and panel settings
ES|QL field missing	Incorrect mapping or other source selected	Check _mapping and index name
Formula displays 0.417	Number format rather than Percent	Set Lens value format to Percent

Key acceptance standard

A dashboard passes validation when its totals, grouped counts, ratios, and filter behavior all match the independently queried training data in the exact same time range.

10. Capstone - Operational incident assessment

Participant task

Assume an incident commander asks for a concise update. Use only the sample data and screenshots generated during the labs. Distinguish evidence, impact, hypothesis, and next diagnostic action.

ASSESSMENT QUESTION	EVIDENCE-BASED ANSWER
When did failures start?	6 Nov 2026 at 07:43 IST (02:13 UTC).
Which service failed most?	Payment-service: 4 of the 5 failed requests.
Which host has most failures?	app-02: 3 failed requests.
Were multiple services affected?	Yes. Payment: 4 failures; checkout: 1 failure.
Is there trace evidence?	Yes. trace-pay002 appears on payment and checkout.
What is root cause?	Undetermined; payment dependency fault is a hypothesis.
Is the dashboard accurate?	Confirm only after completing V01-V07.

Sample incident report statement

Incident update - example

Between 07:43 and 07:50 IST, five server-error responses were observed in the 12-event training sample. Payment-service accounted for four failures; checkout-service accounted for one. Three failures originated on app-02 and one on app-03; the remaining failure was on app-01. A shared trace links one payment 503 response with a subsequent checkout 502. Root cause is not yet confirmed. Validate the underlying dependency and inspect metrics, traces, and recent deployments.

Assessment rubric

CRITERION	WEIGHT	PASS EVIDENCE
KQL/ES|QL query accuracy	30%	Queries and expected counts match
Incident interpretation	25%	Timeline, impact, hypothesis separated
Dashboard construction	25%	Required panels, time filter, controls
Independent validation	20%	Dev Tools cross-check + evidence register

Completion criteria: save the dashboard, submit the completed validation checklist and evidence register, and write a short incident summary with an explicit statement of remaining uncertainty.

11. Extension - Apply to your Ubuntu ELK stack

After validating the synthetic workshop, replace the training source with operational logs or metrics already available in your environment. The field names and index or data-stream selection may differ.

SOURCE	POSSIBLE INDEX / DATA STREAM	USE IN INVESTIGATION
Filebeat	filebeat-* or logs-*	Application, system, Apache logs
Logstash custom pipeline	custom index naming	Parsed CSV, syslog, or HTTP poller events
Metricbeat / Elastic Agent	metricbeat-* or metrics-*	CPU, memory, process, JVM, filesystem
APM	traces-apm* (deployment dependent)	Services, latency, traces, span correlation
Heartbeat / Synthetics	heartbeat-* or synthetics-*	Endpoint checks and uptime

Production investigation checklist

- Confirm the target index/data stream has recent records and that the time picker matches your local timezone.
- Inspect **available fields** before writing KQL; the training fields may not exist in production records.
- Check whether records represent **requests**, **log entries**, or **metrics**; avoid treating raw log counts as request counts without suitable instrumentation.
- Correlate an APM trace ID, if present, across related service logs; consider sampling and missing spans.
- Compare error counts with transaction volume, deployments, node/resource metrics, upstream failures, and alert history.
- For actual service-level indicators, define the numerator, denominator, window, data freshness, and missing-data behavior.

Version and permissions notes

Dashboard, Lens, and Discover labels may differ across versions. Data view creation requires appropriate Kibana and Elasticsearch privileges. Some observability features or data streams depend on which integrations are installed. **This lab does not require Fleet, APM, or Metricbeat to be newly installed.**

Official product documentation

Data views: https://www.elastic.co/docs/explore-analyze/find-and-organize/data-views/create-data-view
Lens & formulas: https://www.elastic.co/docs/explore-analyze/visualize/lens
Discover with ES|QL: https://www.elastic.co/docs/explore-analyze/discover/use-esql
ES|QL CASE: https://www.elastic.co/docs/reference/query-languages/esql/functions-operators/conditional-functions-and-expressions/case
Dashboard filters & inspect: https://www.elastic.co/docs/explore-analyze/dashboards/using

Manual prepared for training use. The incident and hostnames are simulated; no production incident is claimed.